재무 시뮬레이터 구현 명세서

Oct 12, 2026 · @민석 · 캡스톤디자인 6조

0. 범위와 읽는 법

이 문서는 Spring Boot 백엔드에 계산 엔진과 분석을 옮길 때 필요한 계산 순서·수식·변수·입출력 변환을 한곳에 모은 것이다. 이 문서와 정답 파일만으로 구현과 검증이 끝나는 것을 목표로 한다.

무엇을 볼 때	어디를 보나
HTTP 요청·응답 형식, 오류 코드	API 명세서 (수정본)
값의 근거·등급, 검증 결과	설계 보고서 5·7장
계산 순서·수식·변환·분석 정의	이 문서
정답	`reference_python/kernel.py` v1.6, `engine/engine.js` (둘은 같은 결과)

문서와 코드가 다르면 코드가 정답이다. 그때는 이 문서를 고친다.

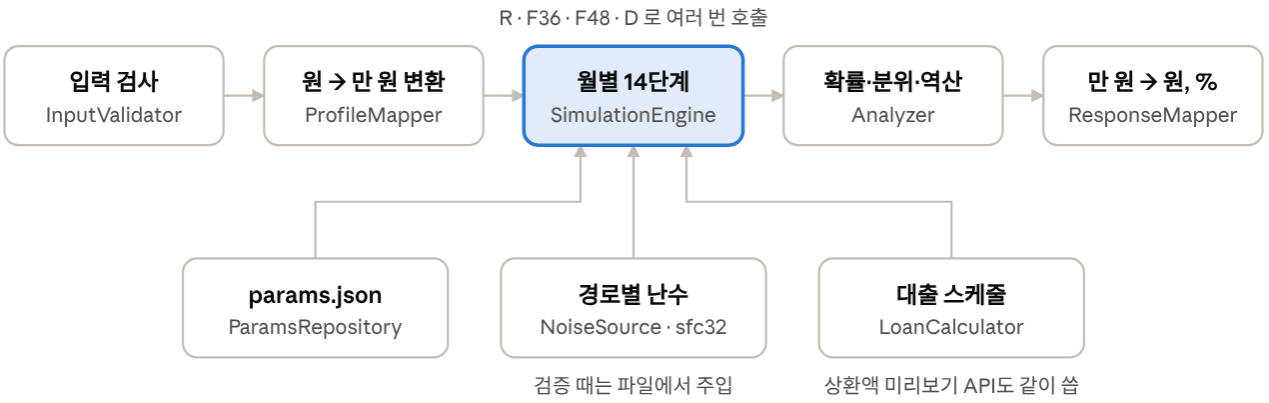
단위와 표기

- 엔진 안의 금액은 **만 원 단위 double**이다. API 경계에서 원 ÷ 10,000으로 넣고, 결과는 × 10,000 후 원 단위 HALF_UP으로 돌려준다.
- 금리·비율은 엔진 안에서 **연 소수**(3.03% → 0.0303)다.
- t는 0부터 세는 달이다. t = 0이 기준 연월(startMonth)이다. 모은 돈 기록 saved[m]은 m = 0~T로 T + 1개다 (saved[0] = 시작 시점).
- p는 경로 번호(0 ~ N − 1)다. 기본 N = 2,000, T = 120, seed = 20261007.
- 코드 이름은 engine.js 기준으로 적고, 필요할 때 kernel.py 이름을 괄호로 붙인다.

1. 전체 구조

요청 하나는 검사 → 변환 → 엔진(여러 번) → 분석 → 응답 변환 순서로 처리한다. 엔진은 상태가 없는 순수 함수다. 같은 프로필·파라미터·난수면 항상 같은 결과를 낸다 (NFR-02).

엔진은 한 요청에 여러 번 불리고, 값·난수·대출은 따로 주입된다



요청 처리 흐름 · 모듈 8개

윗줄은 요청 하나가 지나는 순서이고, 아랫줄 세 모듈은 엔진에 값을 넣어 주기만 한다.

모듈 (제안 이름)	하는 일	자세히
InputValidator	API 명세 5.2의 오류·경고 판정	8장
ProfileMapper	SimulationInput(원) → 엔진 프로필(만 원), 이벤트 개월 변환, adjustments 적용	2장
ParamsRepository	params.json을 서버 시작 때 한 번 읽음 (읽기 전용)	3장
NoiseSource	경로별 난수. 운영은 sfc32 구현, 검증은 파일에서 주입	4장
LoanCalculator	대출 상환 스케줄. 상환액 미리보기 API와 엔진이 같이 씁	6장
SimulationEngine	경로마다 매달 14단계 계산	5장
Analyzer	확률·분위·위험·원인·비교·역산·범위·첫 문장	7장
ResponseMapper	만 원 → 원, 비율 → %, 개월 → YYYY-MM	2장

엔진 함수 한 개의 입출력

- 입력: 프로필, 파라미터, NoiseSource, 경로 수 N, 기간 T, 옵션 3개 (`bufferFirst` 비상금 금액, `collect` 매달 기록 여부, `policy` 청년미래적금 규칙)
- 출력: `saved[N][T+1]` (collect일 때만), 경로별 플래그 12종, `firstBrk[N]` (청년미래적금을 처음 해지한 달, 없으면 −1. 일반 적금 해지는 세지 않음), `matchPaid[N]` (실제 받은 지원금), `eligible`

API별 엔진 호출

R = 본 계산(N = 2,000, T = 120, collect 켬). F36·F48 = 첫 계좌 계산(N = 2,000, T = 36·48, policy.rejoin = false, collect 끔). D = 확정 계산(난수 0, N = 1, T = 120, collect 켬). 네 호출 모두 R과 같은 seed·bufferFirst를 쓰고, D는 R과 같은 policy를 쓴다.

API	엔진 호출
POST /api/simulate	R + F36 + F48 + D
POST /api/simulate/preview	지금 계획과 바꾼 계획 각각 R + F36 + F48
POST /api/compare	지금 계획 + 계획 1~3개 각각 R + F36 + F48
POST /api/reverse	지금 계획 R, 격자 탐색 R 여러 번, 비상금 전략 5종 각각 R + F36 + F48, 개선 포인트 최대 5개 R
POST /api/robust	60번 × (파라미터를 바꾼 R, T = 목표 개월), 비교 계획이 있으면 계획마다 같은 횟수
POST /api/reports	simulate + reverse의 개선 포인트 + compare
나머지 (validate, repayment-preview, meta, presets, examples)	엔진 호출 없음

2. 입력 변환과 출력 변환

엔진은 API 입력(SimulationInput)을 직접 받지 않는다. ProfileMapper가 아래 표대로 바꾼 엔진 프로필만 받는다. 금액은 원 ÷ 10,000, 금리는 % ÷ 100이다.

2.1 입력 → 엔진 프로필

API 필드	엔진 변수	변환
startMonth	(t = 0)	모든 연월을 개월 수로 바꾸는 기준
person.age	age	그대로
person.careerYears	exp_years	그대로
person.employmentType	employment	REGULAR → 정규, CONTRACT → 계약 (없으면 정규)
income.monthlyNetIncome	net_income	÷ 10,000
expenses.fixed / variable	fixed / variable	합계 ÷ 10,000
assets.cash + assets.deposits	cash0	둘을 더한 뒤 ÷ 10,000

API 필드	엔진 변수	변환
assets.investments	invest0	÷ 10,000, 없으면 0
savingPlan.monthlySaving	save	÷ 10,000
savingPlan.generalSavingRate	general_rate	÷ 100, null이면 0.0303
savingPlan.monthlyInvestment	invest_monthly	÷ 10,000, 없으면 0
emergency.breakThreshold	thr	÷ 10,000, 없으면 50 (= 50만 원)
loans.items[]	loans[]	principal ÷ 10,000 → balance, annualRate ÷ 100 → rate, remainingMonths → months, repaymentType → method (null이면 아래 종류별 기본값)
goal.targetAmount	goal	÷ 10,000
goal.targetDate	gm (목표 개월)	startMonth부터 센 개월 차이 (12~120)
events[]	events[]	아래 2.2

상환 방식: EQUAL_PAYMENT → annuity, EQUAL_PRINCIPAL → equal_principal, BULLET → bullet, INTEREST_ONLY → interest_only.

loanType	annualRate가 null이면	repaymentType이 null이면
STUDENT	1.7	EQUAL_PAYMENT
JEONSE	4.3	INTEREST_ONLY
CREDIT	6.3	EQUAL_PAYMENT
MORTGAGE	4.7	EQUAL_PAYMENT
OTHER	5.0	EQUAL_PAYMENT

금리 ÷ 100은 double 나눗셈 그대로다 (3.03 ÷ 100 = 0.030299999999999997로, 기본값 0.0303과 마지막 자리가 다르다. 정답 파일도 이 값이다).

지출 합계는 inputMode가 DETAIL이면 items[].amount의 합, TOTAL이면 total을 쓴다 (제안: 둘이 다르면 items 합을 믿는다).

2.2 예정된 일 → 엔진 이벤트

모든 이벤트의 t = startMonth부터 occurDate까지의 개월 수(0~119)다. 한 달에 여러 개가 있어도 된다.

eventType	엔진 이벤트
ONE_TIME {amount}	{t, kind: lump, amount: amount ÷ 10,000}
SPEND_CHANGE {monthlyChange}	{t, kind: spend, delta: monthlyChange ÷ 10,000} (음수 가능)
JOB_CHANGE {incomeChangeRate, gapMonths}	{t, kind: income, pct: incomeChangeRate ÷ 100, gap: gapMonths}
MOVE {deposit, monthlyChange, amount}	{t, kind: move, deposit: deposit ÷ 10,000, delta: monthlyChange ÷ 10,000, cost: amount ÷ 10,000}

프리셋 5종(독립·전세 이사·결혼·이직·자동차)은 프론트가 기본값을 채운 보통 이벤트로 들어오므로 서버에 따로 분기가 없다. 자동차는 ONE_TIME + SPEND_CHANGE 두 개다.

2.3 adjustments (미리보기·비교·범위)

덮어쓸 값만 온다. null인 항목은 원래 입력을 쓴다.

adjustments 필드	바뀌는 것
monthlySaving	save
monthlyVariableExpense	variable
monthlyInvestment	invest_monthly
goalTargetDate	gm
emergencyFirstAmount	엔진 옵션 bufferFirst (÷ 10,000). 0이면 끔
events	events[] 목록 전체 교체

2.4 엔진 결과 → 응답

- 금액: new BigDecimal(x × 10000).setScale(0, HALF_UP)의 Long (x는 만 원 double).
- 확률: 적중 경로 수(정수)로 `BigDecimal(hits × 100) ÷ N`을 소수 1자리 HALF_UP. double로 계산하지 않는다 (0.2885 × 100 = 28.8499… 문제).
- 차이(%p): 반올림 전 적중 수의 차로 계산한 뒤 반올림한다.
- 오차: p = hits / N(반올림 전)으로 √(p(1−p)/N) × 100을 소수 1자리 HALF_UP. 해지 확률·빚질 확률·원인 비율도 개수(정수)에서 확률과 같은 방식으로 반올림한다 (원인 비율 = BigDecimal(개수 × 100) ÷ 네 개수의 합).

- z: 소수 2자리 HALF_UP (API 예시 16.86과 같은 자릿수). 오차·z·가능성 범위처럼 double에서 반올림하는 값은 모두 BigDecimal.valueOf(v).setScale(k, HALF_UP)이다. new BigDecimal(v)를 쓰면 98.05(실제 double 98.0499…)가 98.0이 되어 정답과 다르다. 금액만 new BigDecimal(x × 10000)이다.
- 달: 개월 m → `startMonth.plusMonths(m)` (YearMonth).

3. 파라미터와 상수

엔진이 쓰는 값은 아래 3묶음(PARAMS·POLICY·GENERAL)이 전부다. 사용자는 이 값을 바꿀 수 없고, 가능성 범위 계산(7.5)만 PARAMS의 복사본을 바꾼다. ParamsRepository는 params.json의 params·policy·general을 이 키 이름 그대로 읽는다. 같은 파일에 근거(meta.parameters, GET /api/meta), 대출 기본값(loanDefaults), 예정된 일 프리셋(eventPresets), 가능성 범위의 범위·급전 확률표(robust)가 있다. 파일은 params/build_params.py가 kernel.py에서 만들고, params/check_params.js가 엔진 값 일치·정답 파일 재현·근거 완비를 검사해 params/validation_report.md를 쓴다. 금액은 만 원, 비율은 소수다.

3.1 PARAMS (통계)

키	값	뜻
inflation.mu_y	0.0208	연평균 물가 상승률
inflation.sigma_y	0.0132	연 물가의 표준편차
inflation.phi	0.55	전년 물가가 올해에 남는 정도 (AR(1))
inflation.last	0.021	계산 시작 전년 물가
wage.macro_mu	0.0285	모두에게 같은 연 임금 상승률
wage.macro_sd	0.0115	그 변동
wage.idio_sd	0.115	개인별 임금 변동
wage.rho_macro	0.65	임금 변동과 전년 물가 충격의 상관
wage.exp_edges	[2, 4, 7.5]	경력 구간 경계(년)
wage.exp_prem	[0.0652, 0.0515, 0.0445, 0.0401]	경력 구간별 추가 상승률
unemp.age_edges	[25, 30, 35]	나이 구간 경계
unemp.u_by_age	[0.088, 0.074, 0.037, 0.025]	나이 구간별 실업률 (20~24 / 25~29 / 30~34 / 35~39)

키	값	뜻
unemp.exit_m	0.092	실직 중 매달 다시 일할 확률
unemp.invol_share	0.274	실직 중 비자발 비율
unemp.contract_ratio	12.3 / 9.9	계약직 실직 위험 ÷ 정규직 실직 위험
unemp.contract_share_pop	0.34	전체 근로자 중 비정규 비율
unemp.scar_per_year	0.067	공백 1년당 재취업 임금 손실
ui.floor / ui.cap	198.144 / 204.3	실업급여 월액 하한 / 상한
ui.replace	0.60	실업급여 = 평균임금의 60%
ui.waiting_frac	7 / 30	첫 달 대기 7일 차감
ui.gross_over_net	1 / 0.88	세전 ÷ 실수령
ui.days_edges / ui.days	[1, 3, 5, 10] / [120, 150, 180, 210, 240]	근무 연수별 실업급여 일수
ui.min_insured_m / ui.window_m	6 / 18	최근 18개월 중 6개월 이상 근무해야 수급
spend.raise_share	0.45	늘어난 실질소득 중 지출로 가는 비율
spend.unemp_cut_ui / unemp_cut_noui	0.068 / 0.22	실직 중 생활비 감소 (실업급여 받음 / 못 받음)
shock.prob_m	0.0482	매달 급전 확률
shock.amounts	[(120, 0.70), (300, 0.30)]	급전 금액과 비율 (물가 반영 전)
overdraft.annual_rate	0.0531	마이너스 금리
liquid.annual_rate	0.0068	파킹통장 금리

3.2 POLICY (청년미래적금)

키	값	뜻
cap	50	월 납입 한도
rate	0.050	기본금리 (입금건별 단리)
term	36	만기 개월

키	값	뜻
match / match_cap	0.06 / 3.0	지원금 = min(납입액 × 6%, 3만 원)
window	6	퇴직 후 이 개월 안 해지는 특별해지
tax	0.154	이자소득세 (일반해지 이자에만)
age_max / gross_max_y	34 / 7500	가입 자격: 만 34세 이하, 총급여 7,500만 원 이하
rejoin / rejoin_wait	true / 2	해지 후 재가입, 해지월 기준 2개월 뒤
early_table	[(3, 0.5), (6, 0.7), (9, 0.8), (11, 0.9)]	보유 개월 하한별 중도해지 이율 비율
early_floor / early_under3	0.0015 / 0.001	중도해지 이율 최저 / 3개월 미만 고정 이율

3.3 GENERAL (일반 적금)

키	값	뜻
rate	0.0303	기본 금리 (사용자 입력 general_rate가 있으면 그 값)
early_ratio	0.5	중도해지 때 받는 이자 비율
tax	0.154	이자소득세

12.3 / 9.9, 7 / 30, 1 / 0.88은 나눗셈 결과를 double로 그대로 쓴다. 소수 몇 자리로 잘라 적으면 정답 파일과 1e−9 안에서 맞지 않는다.

4. 난수

Java는 engine.js의 난수 생성기(sfc32)를 비트 단위로 그대로 옮긴다. 그러면 Java 결과가 데모·API 명세 예시 값(S1 58.0% 등)과 같아져, 예시 값을 그대로 인수 시험에 쓸 수 있다. java.util.Random이나 SplittableRandom을 쓰면 확률이 몬테카를로 오차(±1%p)만큼 달라져 예시와 대조할 수 없다.

4.1 경로 하나가 쓰는 난수

noise(p, T)는 경로 p의 난수 배열 7개를 돌려준다. Y = ⌊T / 12⌋ + 1.

배열	길이	분포	쓰는 곳
z1	T	표준정규	물가 충격, 임금의 물가 연동 부분

배열	길이	분포	쓰는 곳
z2	Y	표준정규	임금의 개인 변동
u_onset	T	[0, 1) 균등	실직 발생
u_dur	T	[0, 1) 균등	공백 기간
u_invol	T	[0, 1) 균등	비자발 여부
u_shock	T	[0, 1) 균등	급전 발생
u_amt	T	[0, 1) 균등	급전 금액

- **대조 변량**: N이 짝수면 half = N / 2. 경로 p의 정규분포는 k = p mod half의 스트림에서 뽑고, p ≥ half이면 부호를 뒤집는다. 균등분포는 경로마다 따로 뽑는다. N이 홀수면 대조 변량을 쓰지 않는다.
- **뽑는 순서**: 정규 스트림에서 z1 T개 → z2 Y개. 균등 스트림에서 u_onset T개 → u_dur → u_invol → u_shock → u_amt.
- **T에 따라 달라진다**: 순서대로 뽑기 때문에 T = 36인 F36의 z2·u_dur 등은 T = 120인 R과 다른 값이다. 엔진 호출마다 그 호출의 T로 noise(p, T)를 다시 만든다. R의 난수를 잘라 쓰면 안 된다.
- **정규분포**: Box–Muller 한 쌍 중 하나만 쓴다. u1 = 스트림에서 0이 아닌 값이 나올 때까지 뽑고, u2는 다음 값, z = √(−2 ln u1) × cos(2π u2).

4.2 Java 구현 (engine.js 55~74행과 1:1)

int 연산은 JavaScript의 `| 0`, `Math.imul`, `>>>`과 비트가 같다. ln·cos는 StrictMath를 쓴다 (JS 엔진과 같은 fdlibm 계열이라 값이 같을 가능성이 높다. 다르더라도 마지막 자리 차이다).

```
final class Sfc32 {
    private int a, b, c, d;
    Sfc32(int a, int b, int c, int d) { this.a = a; this.b = b; this.c = c;
this.d = d; }
    double next() {
        int t = a + b + d;
        d = d + 1;
        a = b ^ (b >>> 9);
        b = c + (c << 3);
        c = (c << 21) | (c >>> 11);
        c = c + t;
        return (t & 0xFFFFFFFFL) / 4294967296.0;
    }
    static int mix(int seed, int k, int s) {
        int h = seed ^ 0x9e3779b9;
        for (int v : new int[]{k, s}) {
            h = (h ^ (v + 0x7f4a7c15)) * 0x85ebca6b;
            h ^= h >>> 13;
```

```
            h = h * 0xc2b2ae35;
            h ^= h >>> 16;
        }
        return h;
    }
    static Sfc32 of(int seed, int k, int s) {
        Sfc32 r = new Sfc32(mix(seed, k, s), mix(seed, k, s + 11), mix(seed,
k, s + 23), mix(seed, k, s + 37));
        for (int i = 0; i < 12; i++) r.next();   // 처음 12개는 버린다
        return r;
    }
}

// 경로 p의 난수 (N, seed는 호출하는 쪽에서 고정)
int half = (N % 2 == 0) ? N / 2 : N;
int k = p % half;
double sign = (N % 2 == 0 && p >= half) ? -1 : 1;
Sfc32 rn = Sfc32.of(seed, k, 1);   // 정규분포용: 짝 경로가 같은 스트림
Sfc32 ru = Sfc32.of(seed, p, 2);   // 균등분포용: 경로마다 다른 스트림
// gauss(): double u = 0; while (u == 0) u = rn.next();
//          return StrictMath.sqrt(-2 * StrictMath.log(u)) * StrictMath.cos(2
* Math.PI * rn.next());
// z1[j] = sign * gauss() (j < T) → z2[j] = sign * gauss() (j < Y)
// u_onset[j] = ru.next() (j < T) → u_dur → u_invol → u_shock → u_amt
```

4.3 확인용 값 (seed 20261007, N = 2,000, T = 120)

Java 구현이 아래 값을 내면 난수는 맞다. 테스트는 Double.parseDouble(표의 값) == 계산값으로 비교한다 (문자열 비교는 자릿수 표기가 달라 틀린다). 값은 engine.js로 뽑았고, 4.2의 Java 코드(Java 21)로 모두 같은 값이 나오는 것을 확인했다.

항목	값
p = 0, z1[0..2]	−0.57882532845290124, 1.1554114528094823, 0.37425810223580386
p = 1000, z1[0..2]	0.57882532845290124, −1.1554114528094823, −0.37425810223580386 (p = 0과 부호만 반대)
p = 0, z2[0]	0.44112529436245818
p = 0, u_onset[0..1]	0.56416278285905719, 0.048801576253026724
p = 0, u_amt[119]	0.43579852511174977
p = 1000, u_onset[0]	0.51368772634305060

항목	값
T = 36일 때 p = 0, z2[0]	−0.59076490877080257 (T = 120과 다름)

4.4 특수한 난수

- **확정 계산(난수 0)**: z1 = z2 = 0, u_onset = 1, u_dur = 0, u_invol = 1, u_shock = 1, u_amt = 0. 실직·급전이 없고 물가·임금은 평균값으로 간다.
- **정답 파일 대조용**: golden.json의 noise(파이썬 PCG64로 만든 값)를 그대로 주입한다. 그래서 엔진은 난수를 직접 만들지 말고 NoiseSource 인터페이스로 받는다 (8장).
- **가능성 범위**: j번째 시도는 seed = 900 + j로 만든다 (7.5).

5. 월별 계산 순서와 수식

경로마다 상태 변수 32개를 두고, t = 0부터 T − 1까지 매달 아래 14단계를 이 순서로 계산한다. 순서가 바뀌면 결과가 달라진다. 단계 번호는 보고서 6.1과 같다. 경로끼리는 서로 영향이 없어 경로 단위로 병렬 처리해도 된다.

5.1 계산 전에 한 번 (프로필별)

아래에서 age0 = age, exp0 = exp_years, net0 = net_income(프로필 값)이다.

- incomeOk = net0 × 12 × gross_over_net ≤ gross_max_y + 1e−6
- eligible = age0 ≤ age_max 그리고 incomeOk
- a1 = eligible이면 min(save, cap), 아니면 0 (매달 청년미래적금에 넣을 금액)
- loanPay[t] = 모든 대출의 t월 상환액 합 (6.1)
- events = t 오름차순 안정 정렬
- epsSd = sigma_y × √(1 − phi²)
- sig = √(macro_sd² + idio_sd²) (`Math.hypot`), rho = rho_macro × macro_sd ÷ sig (sig = 0이면 0)
- gRate = general_rate (없으면 GENERAL.rate)
- rOd = overdraft.annual_rate / 12, rL = liquid.annual_rate / 12, rP = POLICY.rate / 12, rG = gRate / 12
- uiAmt = min(max(replace × net0 × gross_over_net, floor), cap)

5.2 상태 변수와 시작값 (경로별)

변수	뜻	시작값
L	파킹통장	cash0

변수	뜻	시작값
od	마이너스 잔액	0
INV	투자 자산	invest0
DEP	보증금 자산	0
P1 / C1	청년미래적금 원리금 / 원금	0 / 0
M1 / Mp	지원금 원리금 / 지원금 원금	0 / 0
held	지금 계좌 보유 개월	0
cumHeld	재가입까지 합친 누적 가입 개월	0
act1	청년미래적금 가입 중	a1 > 0
matchMult	지원금 비율 (재가입 때 줄어듦)	1
wait	재가입까지 남은 달 (−1 = 대기 없음)	−1
G / CG	일반 적금 원리금 / 원금(재예치 후 원리금 포함)	0 / 0
gHeld	일반 적금 이번 회차 개월	0
started	비상금 조건을 채웠는가	L ⩾ bufferFirst 또는 bufferFirst ⩽ 0
base	재직 때 월 소득 (연봉 인상·재취업 손실 반영)	net0
incomeMult	이직으로 바뀐 소득 배수	1
cum	누적 물가 지수	1
piPrev / mInf	전년 물가 / 올해 월 물가	inflation.last / 0
unemp	남은 공백 개월 (0 = 재직)	0
uiLeft / uiFirst	남은 실업급여 개월 / 실업급여 첫 달 여부	0 / false
since	공백 시작 후 지난 달	1,000,000
gapLen	이번 공백 길이	0
volGap	이직 공백 (이번 달 적용 대기)	0
lastShock	마지막 급전 달	−99
extraFixed	이벤트로 바뀐 월 지출	0
insured	최근 18개월 중 근무 개월	min(exp0 × 12, 18)

변수	뜻	시작값
mp	실제 받은 지원금 누계 (→ matchPaid)	0

saved[0] = L + P1 + G + INV + DEP − od.

5.3 매달 14단계

ageNow = age0 + t / 12, expNow = exp0 + t / 12 (둘 다 double).

1. **연초** (t mod 12 = 0일 때만)
   - n = min(12, T − t), Zy = (z1[t] + … + z1[t + n − 1]) ÷ √n
   - π = mu_y + phi × (piPrev − mu_y) + epsSd × Zy, mInf = (1 + π)^(1/12) − 1, piPrev = π
   - t > 0이면 연봉 인상: Z1 = (z1[t − 12] + … + z1[t − 1]) ÷ √12, g = macro_mu + expPrem(expNow − 1) + sig × (rho × Z1 + √(1 − rho²) × z2[t / 12 − 1]), base = base × (1 + g)
2. **물가 누적**: cum = cum × (1 + mInf)
3. **예정 이벤트**: lump = 0, depAdd = 0으로 두고 t가 같은 이벤트를 순서대로 적용
   - lump: lump += amount
   - spend: extraFixed += delta
   - move: depAdd += deposit, extraFixed += delta, lump += cost
   - income: incomeMult = incomeMult × (1 + pct). gap > 0이고 unemp = 0이면 volGap = gap
4. **고용**
   - h = hazard(employment, ageNow) (5.4)
   - start = (unemp = 0) 그리고 (volGap = 0) 그리고 (u_onset[t] < h)
   - D = geom(u_dur[t], exit_m, T), invol = u_invol[t] < invol_share
   - start이면: unemp = D, gapLen = D, since = 0. invol이고 insured ≥ min_insured_m이면 uiLeft = uiMonths(expNow), uiFirst = true. 아니면 uiLeft = 0 (uiFirst는 그대로. uiFirst는 실직 첫 달 13단계에서 항상 false가 되므로 false로 두어도 결과는 같다)
   - volGap > 0이면: unemp = max(unemp, volGap), gapLen = volGap, since = 0. 그다음 volGap = 0
   - employed = (unemp = 0)
   - income = employed이면 base × incomeMult. 아니고 uiLeft > 0이면 uiAmt × (uiFirst이면 1 − waiting_frac, 아니면 1). 둘 다 아니면 0
5. **월초 이자**: od = od × (1 + rOd), L = L × (1 + rL)
6. **급전**: hit = u_shock[t] < prob_m. hit이면 shock = pick(u_amt[t]) × cum, lastShock = t. 아니면 shock = 0

7. **현금 흐름**
   - lifestyle = raise_share × max(base × incomeMult − net0 × cum, 0) (실직 중에도 같은 식)
   - cut = employed이면 0, 아니고 uiLeft > 0이면 unemp_cut_ui, 아니면 unemp_cut_noui
   - living = ((fixed + variable + extraFixed) × cum + lifestyle) × (1 − cut)
   - L = L + income − living − shock − lump − loanPay[t] − depAdd, DEP = DEP + depAdd
8. **마이너스 상환**: rep = min(max(L, 0), od), L −= rep, od −= rep
9. **재가입 → 납입 → 적금 이자**
   - 재가입 (policy.rejoin일 때만): wait > 0이면 wait −= 1. 그 뒤 wait = 0이고 ageNow ⩽ age_max이고 incomeOk이고 cumHeld < term이면: act1 = true, held = 0, matchMult = (term − cumHeld) ÷ term, wait = −1, 플래그 rejoin
   - started = started 또는 (L ⩾ bufferFirst), can = employed 그리고 started
   - 청년미래적금: put1 = (can 그리고 act1)이면 min(a1, max(L, 0)), 아니면 0. L −= put1, P1 += put1, C1 += put1, mNew = min(match × put1, match_cap) × matchMult, M1 += mNew, Mp += mNew
   - 일반 적금: rest = act1이면 save − a1, 아니면 save. putG = can이면 min(rest, max(L, 0)), 아니면 0. L −= putG, G += putG, CG += putG
   - 투자: putI = employed이면 min(invest_monthly, max(L, 0)), 아니면 0 (비상금 조건과 무관). L −= putI, INV += putI
   - 청년미래적금 이자 (act1일 때): P1 += C1 × rP, M1 += Mp × rP
   - 일반 적금 이자: CG > 0이면 G += CG × rG, gHeld += 1. gHeld ⩾ 12이면 CG = CG + (G − CG) × (1 − tax), G = CG, gHeld = 0. CG = 0이면 gHeld = 0
10. **만기**: act1이면 held += 1, cumHeld += 1. 그 뒤 act1이고 held ⩾ term이면: mp += M1, L += P1 + M1, P1 = C1 = M1 = Mp = 0, act1 = false, 플래그 mature1 (만기 뒤에는 재가입하지 않고 저축은 모두 일반 적금으로 간다)
11. **부족분**: deficit = max(−L, 0), od += deficit, L = max(L, 0). deficit > 0이면 플래그 overdraft
12. **해지** (일반 적금 먼저)
    - od > thr이고 G > 0이면: prG = CG + (G − CG) × early_ratio × (1 − tax), r2 = min(prG, od), od −= r2, L += prG − r2, G = CG = gHeld = 0, 플래그 brkG
    - 그래도 act1이고 od > thr이면 청년미래적금 해지: sp = (since ⩽ window). 받는 돈 pr1 = sp이면 P1 + M1(그리고 mp += M1), 아니면 C1 + (P1 − C1) × earlyRatio(held) × (1 − tax). r3 = min(pr1, od), od −= r3, L += pr1 − r3. P1 = C1 = M1 = Mp = 0, act1 = false, wait = rejoin_wait
    - 이것이 이 경로의 첫 해지면 firstBrk = t, 원인 플래그 하나: sp이면 c1_sp, 아니고 실직 중이면 c1_unemp, 아니고 t − lastShock ⩽ 2이면 c1_shock, 나머지는 c1_deficit
    - 해지할 때마다 sp이면 brk1_sp, 아니면 brk1_gen 플래그
    - 끝에 od > thr이면 플래그 limit. 투자 자산과 보증금은 쓰지 않는다

13. **카운터**
    - 실직 중이고 unemp = 1(이번 달이 공백 마지막)이면 base = base × max(1 − scar_per_year × gapLen / 12, 0.5)
    - insured = employed이면 min(insured + 1, window_m), 아니면 max(insured − 1, 0)
    - unemp = max(unemp − 1, 0)
    - 실직 중이었으면 uiLeft = max(uiLeft − 1, 0), uiFirst = false
    - since += 1
14. **기록**: saved[t + 1] = L + P1 + G + INV + DEP − od (대출 잔액은 빼지 않는다)

경로가 끝나면 matchPaid[p] = mp. 플래그 12종(unemp, overdraft, brk1_sp, brk1_gen, brkG, mature1, limit, rejoin, c1_sp, c1_unemp, c1_shock, c1_deficit)은 한 번이라도 켜지면 1이다. unemp 플래그는 4단계에서 실직 중이면 켜다.

5.4 도우미 함수

- **sr(edges, x)** = edges 중 x 이하인 경계의 개수 (numpy `searchsorted(side="right")`). 구간 번호로 쓴다.
- **hazard(emp, age)**: u = u_by_age[sr(age_edges, age)], h = u × exit_m ÷ (1 − u). c = contract_share_pop, r = contract_ratio, m = 1 ÷ ((1 − c) + c × r). 계약직이면 h × m × r, 정규직이면 h × m.
- **expPrem(y)** = exp_prem[sr(exp_edges, y)].
- **uiMonths(tenure)** = round(days[sr(days_edges, tenure)] ÷ 30) → 4~8개월.
- **geom(u, e, cap)** = min(1 + ⌊ln(min(max(1 − u, 1e−12), 1)) ÷ ln(1 − e)⌋, cap). 공백은 최소 1개월이고 cap은 그 호출의 T다.
- **pick(u)**: 누적 비율 [0.70, 1.00]에서 u 이하인 값의 개수를 인덱스로 쓴다. u < 0.70이면 120, 아니면 300.
- **earlyRatio(held)**: ratio = early_table에서 하한 ⩽ held인 마지막 비율(없으면 0). eff = held < 3이면 early_under3, 아니면 max(rate × ratio × held ÷ term, early_floor). 돌려주는 값 = eff ÷ rate.

6. 대출·이벤트·저축 규칙

청년미래적금과 일반 적금의 납입·이자·만기·해지는 5.3의 9~12단계가 전부다. 이 장은 5.3에서 한 줄로 넘긴 대출 스케줄과, 규칙의 뜻을 모아 적는다.

6.1 대출 상환 스케줄 (LoanCalculator)

대출마다 아래를 계산해 달별로 더한다. 상환액은 실직 중에도 나간다. B = balance, i = rate / 12, n = months.

- A = (annuity이고 i > 0)이면 B × i(1 + i)^n ÷ ((1 + i)^n − 1), annuity이고 i = 0이면 B / n

- t = 0 ~ T − 1: B ⩽ 1e−9이면 건너뛴다. it = B × i로 두고
  - annuity: p = min(A, B + it)
  - equal_principal: p = min(B, 처음 원금 / n) + it
  - interest_only: p = t < n이면 it, 아니면 0. 원금은 줄지 않는다 (보증금 반환으로 갚는다고 보고 현금흐름 밖). t = n − 1이 지나면 B = 0
  - bullet: p = it + (t = n − 1이면 B, 아니면 0)
  - interest_only가 아니면 B = max(B − (p − it), 0)
  - pay[t] += p

상환액 미리보기 API는 같은 함수를 대출 한 건, T = n으로 부른다. firstMonthPayment = pay[0], firstMonthInterest = 원금 × i, firstMonthPrincipal = 두 값의 차, lastMonthPayment = pay[n − 1], totalInterest = Σ pay − 갚은 원금. 각각 원 단위 HALF_UP. S1 학자금(1,200만 원, 1.9%, 96개월)은 134,839 / 19,000 / 115,839 / 134,839 / 944,574원으로 API 예시와 같다 (이 정의로 다시 계산해 확인). 입력 검사의 월 상환액(monthlyLoanPayment)은 모든 대출의 pay[0] 합이다.

6.2 이벤트가 바꾸는 것

이벤트	그 달	이후
lump	파킹통장에서 amount 지출 (물가 반영 안 함)	없음
spend	없음	매달 지출에 delta × cum 추가 (물가 반영, 끝나는 달 없음)
move	보증금은 파킹통장 → 보증금 자산(DEP)으로 이동, 이사 비용은 지출	주거비 변화 delta × cum
income	이직 공백이 있으면 이달부터 gap개월 실직 상태 (실업급여 새로 정하지 않음, since = 0이라 특별해지 사유 성립)	재직 소득 × (1 + pct)

이직 공백은 이미 실직 중인 달에는 적용하지 않는다 (소득 배수만 바뀐다). 이직 공백이 끝날 때도 13단계의 재취업 임금 손실이 적용된다.

6.3 저축이 어디로 가는가

- 매달 save 중 a1(최대 50만 원)은 청년미래적금, 나머지는 일반 적금이다. 청년미래적금을 해지해 재가입을 기다리는 동안과 만기 뒤에는 save 전체가 일반 적금으로 간다.
- 통장에 돈이 모자라면 있는 만큼만 넣는다 (max(L, 0) 상한). 넣지 못한 돈을 다음 달에 채우지 않는다.
- 실직 중에는 납입과 투자가 모두 멈춘다. 적금 이자는 계속 붙는다.

- 비상금 먼저(bufferFirst = B)면 파킹통장이 처음으로 B 이상이 된 달부터 적금 납입을 시작하고, 그 뒤에는 다시 따지지 않는다. 투자는 이 조건과 무관하다.
- 모자라는 돈은 마이너스가 되고, 마이너스가 버틸 금액(thr)을 넘으면 일반 적금 → 청년미래적금 순서로 해지해 갚는다. 투자 자산과 보증금은 헐지 않는다.

7. 분석

분석은 엔진 결과(saved, 플래그, matchPaid)만 읽는다. 아래 정의는 API 명세 예시 값을 만든 코드(`engine/make_analysis_golden.js`)와 데모의 계산과 같다. 금액은 만 원, 확률은 비율로 계산하고 응답 변환은 2.4를 따른다. gm = 목표 개월, goal = 목표 금액, τ = 목표 확률(기본 0.70).

7.1 공통 정의

- **분위수** q(x): 그 달의 saved N개를 오름차순으로 정렬하고 인덱스 min(N − 1, ⌊x × N⌋)의 값. 보간하지 않는다. N = 2,000이면 p5 = 100번째, p10 = 200번째, p50 = 1,000번째, p90 = 1,800번째(0부터).
- **중앙값(목록)**: 오름차순 정렬 후 인덱스 ⌊(n − 1) / 2⌋ (낮은 쪽 중앙값).
- **적중**: hit[p] = saved[p][gm] ⩾ goal. 확률 = Σ hit ÷ N.

7.2 시뮬레이션 결과 (/simulate)

응답 필드	계산	엔진 호출
goal.probability	Σ hit ÷ N	R
goal.probabilityError	√(p(1 − p) / N)	R
goal.medianReachDate	경로마다 saved ⩾ goal인 첫 m(0~120, 없으면 ∞)을 N개 모두 모아 중앙값. ∞이면 null	R
goal.reach70Date	#{p: saved[p][m] ⩾ goal} ÷ N ⩾ 0.70인 첫 m(1~120). 없으면 null	R
checkpoints	m = 12, 36, 60, 120의 q(0.05), q(0.10), q(0.50), q(0.90)	R
bands	m = 0~120 전부 같은 4개 분위	R
deterministic.atGoal / at120	D.saved[gm] / D.saved[120]	D
risks.debt.probability	1 ⩽ m ⩽ gm 에서 한 번이라도 saved < 0인 경로 비율	R

응답 필드	계산	엔진 호출
risks.debt.medianFirstDate	그런 경로들의 첫 음수 m의 중앙값. 없으면 null	R
risks.debt.worstCaseAtGoal	gm의 q(0.05)	R
risks.policySaving.eligible	5.1의 eligible	—
risks.policySaving.earlyTerminationProbability	brk1_gen이 켜진 경로 비율. 비대상이면 null	F36
risks.policySaving.expectedSubsidy	matchPaid의 경로 평균. 비대상이면 null	F48
causes	c1_sp → unemployment, c1_unemp → prolongedUnemployment, c1_shock → emergencyExpense, c1_deficit → chronicDeficit. 각 개수 ÷ 네 개수의 합 (합이 0이면 모두 0)	F36

F36·F48은 policy.rejoin = false, bufferFirst는 R과 같은 값이다. S1 확인값: probability 58.0, medianReachDate 55개월(2031-06), reach70Date 73개월(2032-12). reach70Date는 기준 구현대로 1개월부터, medianReachDate는 0개월부터 센다. 둘은 시작부터 목표 이상인 입력(WARN_GOAL_ALREADY_MET)에서만 달라진다.

7.3 비교 (/simulate/preview, /compare)

모든 계획을 같은 seed로 계산하고, 계획마다 7.2의 값을 구한다. 각 계획의 적중은 그 계획의 gm으로 판정한다.

- **짝지은 검정**: d[p] = hit_바꾼[p] − hit_지금[p], md = Σd / N, sd = √(max(Σd² / N − md², 0) × N / (N − 1)), z = md ÷ (sd / √N) (sd = 0이면 z = 0).
- **의미 있는 차이**: |z| ≥ 1.96 그리고 |md| ≥ 0.02.
- **yearlyP50**: m = 12, 24, … (gm 이하)의 q(0.50).
- 비교표 항목: PROBABILITY, DEBT_PROBABILITY, WORST_CASE_AT_GOAL, EARLY_TERMINATION, EXPECTED_SUBSIDY (7.2의 같은 값). diff = 바꾼 − 지금. 확률형 diff는 개수의 차로 계산해 반올림하고, 금액형 diff는 원으로 반올림한 두 값의 차다. 청년미래적금 비대상이면 EARLY_TERMINATION·EXPECTED_SUBSIDY의 current·adjusted·diff가 모두 null이다.

7.4 역산과 개선 포인트 (/reverse)

surplus(월 여유자금) = net0 − fixed − variable − Σ pay[0]. 탐색은 모두 같은 seed의 R이다.

항목	탐색
requiredVariableExpenseCut	k = 5, 10, …, min(variable, 60)만 원을 차례로 줄여 확률 ⩾ τ인 첫 k. 없으면 amount·probability 모두 null. 지금 확률이 이미 ⩾ τ이면 0
requiredMonthlySaving	s = 0, 5, 10, …, surplus 이하에서 확률 ⩾ τ인 가장 작은 s. 없으면 amount = null. maxProbability·그때의 atAmount(같으면 작은 s)·grid(칸마다 금액과 확률)는 항상 돌려주고, surplus < 0이라 격자가 비면 null·null·[]
requiredPeriod	지금 계획 R에서 비율 ⩾ τ인 첫 m(1~120, reach70과 같은 방식, 재계산 없음). 없으면 null
emergencyStrategies	B = 0, 100, 200, 300, 500만 원마다 bufferFirst = B로 R + F36 + F48 → 확률, 해지 확률, 지원금

월 저축액 격자는 납입을 늘리면 해지 위험 때문에 확률이 떨어질 수 있어 이분 탐색을 쓰지 않는다. S1은 0~45만 원에서 확률이 53~58% 안에 머물어 amount = null이 정상이다.

개선 포인트(levers): 아래 후보 중 조건을 채우는 것만, 하나씩 바꿔 R을 돌린다. diff = 확률 − 지금 확률, flat = |적중 수 차| < 0.02 × N, diff 내림차순 안정 정렬.

type	바꾸는 것	조건
REDUCE_VARIABLE_EXPENSE	variable − 10	variable ⩾ 10
DELAY_GOAL	gm + 12	gm + 12 ⩽ 120
LOWER_GOAL	goal → round(goal × 0.9) (만 원)	항상
EMERGENCY_FIRST	bufferFirst = 200	eligible이고 cash0 < 200
REDUCE_SAVING	save − 10	save ⩾ 20

label(화면 문구): REDUCE_VARIABLE_EXPENSE "변동지출을 월 10만 원 줄이기", DELAY_GOAL "목표 시점을 1년 늦추기", LOWER_GOAL "목표 금액을 10% 낮추기", EMERGENCY_FIRST "비상금 200만 원을 먼저 모은 뒤 적금 시작", REDUCE_SAVING "월 저축액을 10만 원 줄이기".

7.5 가능성 범위 (/robust)

K = 60번, 한 번에 M = 2,000경로, T = 각 계획의 gm. 시도마다 기본 PARAMS를 새로 복사해서 바꾼다. j번째 시도는 PARAMS 복사본을 아래 순서로 바꾸고 seed = 900 + j의 난수를 쓴다. 비교 계획은 같은 시도에서 같은 파라미터·난수로 계산한다.

파라미터를 뽑는 난수는 요청마다 한 번 st = 12345로 시작해 60번의 시도 동안 이어 쓰는 mulberry32다: st = st + 0x6D2B79F5, t = (st ^ (st >>> 15)) × (1 | st), t = (t + (t ^ (t >>> 7)) × (61 | t)) ^ t, 값 = ((t ^ (t >>> 14)) 부호 없는 32비트) ÷ 2^{32} (int 곱셈). U(a, b) = a + (b − a) × rnd(), pickOne(배열) = 배열[⌊rnd() × 길이⌋ mod 길이].

1. inflation: mu_y = U(0.0138, 0.0278), sigma_y = U(0.0097, 0.0204), phi = U(0.1, 0.9)
2. wage: macro_mu = U(0.018, 0.038), idio_sd = U(0.006, 0.147), rho_macro = U(0.21, 0.87), exp_prem[3] = U(0.0367, 0.0401)
3. sc = U(0.78, 1), u_by_age의 네 값에 × sc
4. stc = pickOne([long, short]), am = pickOne([0.67, 1.0, 1.5]), pv = pickOne([base, buf3, buf12, spend])
5. exit_m = long이면 0.092, short면 0.241. shock.amounts = [(120 × am, 0.7), (300 × am, 0.3)]. shock.prob_m = 아래 표[stc, am, pv]
6. invol_share = U(0.20, 0.35), scar_per_year = U(0.03, 0.10), raise_share = U(0.24, 0.50)

stc · am	base	buf3	buf12	spend
long · 0.67	0.09358	0.06386	0.1421	0.06998
long · 1.0	0.04817	0.03324	0.07747	0.03681
long · 1.5	0.02733	0.01914	0.04436	0.02133
short · 0.67	0.06882	0.03865	0.12156	0.04469
short · 1.0	0.03552	0.01997	0.06605	0.02341
short · 1.5	0.0202	0.01148	0.03809	0.01358

결과: 60개 확률의 중앙값·2.5%·97.5% 분위. 여기서만 분위수는 선형 보간이다 (h = (n − 1)x, l = ⌊h⌋, s[l] + (s[l + 1] − s[l])(h − l)). 계획이 있으면 시도마다 차이(계획 − 지금)를 구해 같은 분위를 내고, conclusionKept = 기본 비교(7.3)의 차이와 부호(−1·0·+1)가 같은 시도 수. S1 확인값: 66.4 (51.4~83.4), 변동지출 10만 원 감액 차이 11.4 (4.3~15.8), 60번 중 60번 유지.

7.6 첫 문장 (AI 미사용 템플릿)

> 지금 계획이면 {기간} 뒤 {목표}을 모을 가능성은 약 {p}%예요. 계산기로는 {확정}이 나오지만, 실직이나 급한 지출까지 넣으면 그래요.

- {기간}: gm이 12의 배수면 "{gm/12}년", 아니면 "{⌊gm/12⌋}년 {gm mod 12}개월"
- {p}: 화면에 보이는 확률(소수 1자리)을 정수로 HALF_UP (74.7 → 75)

- {목표}, {확정}(D.saved[gm]): 만 원 단위 double x를 BigDecimal.valueOf(x).setScale(0, HALF_UP)으로 반올림해 "4,440만 원", 1억 이상이면 "1억 2,745만 원"(만 단위가 0이면 "1억 원"), 음수는 앞에 "−"(U+2212, 하이픈 아님)
- 조사는 항상 "원"으로 끝나므로 "을", "이"로 고정

가능성 범위 문장: "가정을 60가지로 바꿔 다시 계산하면 대부분 {low}%~{high}% 사이예요. 정확한 한 숫자보다 이 범위를 봐 주세요." (low·high는 화면에 보이는 소수 1자리 값을 정수로 HALF_UP. 96.5 → 97)

8. 검증

Java 구현은 아래 네 단계를 차례로 통과하면 완료다. 1은 엔진 논리, 2는 난수, 3과 4는 분석과 API 전체를 본다. 이 문서만 보고 따로 만든 Java 구현으로 미리 확인했다: 8.1의 12개 case 모두 통과(최대 상대오차 4.5e−12), 4.3 난수 값 일치, 8.4의 분석 정답 8개 입력에서 변환·simulate·preview·compare·reverse·robust 모든 항목 일치.

8.1 엔진 정답 파일 대조 (golden.json)

파이썬 기준 구현이 만든 난수와 결과가 들어 있다. 같은 난수를 주입해 결과가 같은지 본다. 다시 만들 때는 `reference_python/make_golden.py`.

- 구조: N = 100, T = 120, noise(배열 7개, [100][120], z2만 [100][11]), cases 12개
- case: name, pr(엔진 프로필, 만 원·소수 금리, 대출 method가 없으면 annuity), opt(bufferFirst, rejoin), saved[100][121], det[121], flags(12종 [100]), firstBrk[100], match[100]
- 실행: noise(p, T)가 파일의 p번째 행을 돌려주는 NoiseSource로 R(N = 100, policy.rejoin = opt.rejoin ?? true, bufferFirst = opt.bufferFirst ?? 0), 그리고 같은 bufferFirst·policy로 난수 0, N = 1인 D. pr에 없는 키는 invest0 = invest_monthly = 0, loans = events = [], general_rate → GENERAL.rate, thr → 50, employment → 정규
- 통과 기준: saved·det·match의 상대오차 |a − b| ÷ max(1, |b|) ≤ 1e−9, flags·firstBrk 완전 일치. 보고서 8.3 기준(1e−6)보다 엄격하다. JS 엔진은 이 기준에서 최대 4.5e−12로 통과한다 (`node engine/test_engine.js`)

case	확인하는 것
S1	기본 경로, 학자금 원리금균등
S2 비상금200	bufferFirst, 시작 현금 0
S3 이사	move 이벤트, 계약직, 청년미래적금 비대상(36세)
계약·원금균등·만기일시	대출 2건, equal_principal·bullet
이직·결혼·지출변화·투자	income·lump·spend 이벤트, 투자

case	확인하는 것
재가입 없음	policy.rejoin = false, thr = 0, 경력 0.5년
자격 미달 35세	나이 자격
전세대출 이자만+학자금	interest_only
적금 금리 직접 입력 4.5%	general_rate
고소득 자격 미달	소득 자격, 일반 적금 재예치
실업 장기화·버틸 금액 400	실직 중 일반해지(실업 장기화)와 직전 급전이 겹친 경로 (원인 우선순위)
이직 반복·남은 실업급여	이직 4번, 9.2의 남은 실업급여 (100경로 중 13개)

8.2 난수 확인

4.3의 값을 단위 테스트로 둔다.

8.3 API 예시 값 (인수 시험)

sfc32를 그대로 옮겼다면 API 명세의 S1 예시 값이 나와야 한다. 확률은 적중 개수가 같아 정확히 일치해야 하고, 금액은 ±1원 안이면 통과로 본다 (거듭제곱 함수의 마지막 자리 차이). 대상: /simulate(58.0%, 확정 44,396,264원 등), /simulate/preview(70.5%, z 16.86), /compare, /reverse, /robust(66.4%, 51.4~83.4%), /loans/repayment-preview(134,839원).

8.4 분석 정답 파일 (analysis_golden.json)

분석 결과는 `engine/analysis_golden.json`에 있다 (생성: `node engine/make_analysis_golden.js`, 약 80초). 입력 8개마다 API 입력(원 단위)과, 그 입력으로 나와야 하는 /simulate·/simulate/preview·/compare·/reverse·/robust 결과를 API 응답 단위로 담았다. engineProfile은 2장 변환의 결과로, 변환을 디버깅할 때만 쓴다.

case	확인하는 것
S1	API 명세 예시와 같은 값
S2	모은 돈 0원, 3년 목표
S3	청년미래적금 비대상(36세), 이사 이벤트, 버틸 금액 100만원
E1	시작부터 목표 이상, 예금·투자 합산
E2	고소득 비대상, 일반 적금 금리 4.2%

case	확인하는 것
E3	120개월 목표, 이직·자동차, 전세대출 기본값(이자만 4.3%), 투자
E4	지출 > 소득, 저축 0원 (저축액 격자 없음), 신용대출 기본값 (6.3%, 원리금균등)
E5	매달 적자, 확정 계산 음수(첫 문장의 − 표기), 지출 DETAIL 입력

- 비교: 숫자는 2.4 반올림 후 값이 같아야 한다. 확률 중 58처럼 정수로 저장된 것이 있어 숫자로 비교한다 (58 = 58.0). 문자열(headline, label, message)은 글자까지 같아야 한다.
- robust.trialsDetail에 시도별 stc·am·pv와 적중 수가 있어, 범위가 다르면 몇 번째 시도부터 어긋나는지 볼 수 있다.
- 경고(warnings)는 9.3에서 정해지지 않아 넣지 않았다.

엔진 정답 파일에는 해지 원인 우선순위(실직 중 급전)와 9.2 동작을 건드리는 case 2개를 더해 12개가 되었다 (8.1).

8.5 입력 검사

범위·오류 코드는 API 명세 5.1·5.2를 따른다. 만 원으로 바꾼 범위는 `validate_input.py`의 LIMITS와 같다. 경고는 변환한 다음 엔진 단위로 판정한다.

코드	조건 (만 원)
WARN_EXPENSE_OVER_INCOME	fixed + variable ≥ net0
WARN_MONTHLY_SHORTFALL	fixed + variable + Σ pay[0] + save > net0
WARN_GOAL_ALREADY_MET	goal ≤ cash0 + invest0
INFO_POLICY_INELIGIBLE	eligible = false (age0 > 34 또는 net0 × 12 ÷ 0.88 > 7,500)

엔진은 검사를 통과한 입력만 받는다고 가정한다. 그래도 엔진 입구에서 NaN·무한대·음수 금액·months < 1을 다시 막는다 (잘못된 값이 2,000개 경로 전체를 오염시키므로).

9. 구현 시 주의점과 정해야 할 것

9.1 틀리기 쉬운 곳

1. **난수는 호출마다 그 T로 만든다.** F36·F48·가능성 범위(T = gm)에 R(T = 120)의 난수를 잘라 쓰면 결과가 달라진다 (4.1).

2. **분위수 정의가 두 가지다.** 결과·비교는 [x × N]번째 값, 가능성 범위만 선형 보간 (7.1, 7.5).
3. **확률은 적중 개수(정수)에서 BigDecimal로** 반올림한다 (2.4).
4. **나눗셈 상수는 double 그대로** 쓴다 (12.3 / 9.9, 7 / 30, 1 / 0.88).
5. **재가입 확인은 납입 직전**이다. 해지한 달(t)에 wait = 2 → t + 1에 1 → t + 2에 0이 되어 그 달 바로 납입한다.
6. **자격(eligible)은 시작 나이로 한 번** 정한다. 재가입만 그달 나이(ageNow)를 본다.
7. **since의 시작값은 1,000,000**이다. 첫 실직 전에는 특별해지가 생기지 않는다. 0으로 두면 첫 6개월의 해지가 모두 특별해지가 된다.
8. **비교는 엄격한 부등호**다: od > thr, u < h, u < prob_m. 적중만 saved ⩾ goal.
9. **bullet 대출**은 마지막 달에 원금을 통장에서 갚는다 (마이너스가 될 수 있음). interest_only는 원금을 현금흐름 밖에서 갚는다고 본다.
10. **모은 돈에서 대출 잔액을 빼지 않는다** (마이너스만 뺀다). 대출 잔액은 입력 검사의 totalLoanBalance로만 보여준다.
11. **같은 달 이벤트**는 입력 순서를 유지하는 안정 정렬 후 차례로 적용한다.
12. **엔진은 경로 단위로 독립**이다. 병렬로 돌릴 때 결과 배열(saved, 플래그, firstBrk, matchPaid)을 미리 할당하고 각 작업은 자기 경로 인덱스에만 쓴다. 합계·정렬·분위는 모든 경로가 끝난 뒤 한 스레드에서 한다. NoiseSource와 Sfc32는 경로마다 새로 만들어 스레드 간에 공유하지 않는다.
13. **가능성 범위 작업**은 크기가 정해진 스레드 풀에서 돌리고, 결과는 만료(10분)가 있는 동시성 맵에 숫자만 둔다. 입력은 계산이 끝나면 버린다. 요청 본문은 로그에 남기지 않는다 (NFR-06).

9.2 기준 구현에 있는 알려진 동작

짧은 실직(공백 < 실업급여 개월) 뒤 재취업하면 남은 uiLeft가 0으로 돌아가지 않는다. 그 뒤 이직 공백이 오면 그 남은 개월만큼 실업급여를 받고 생활비도 6.8%만 줄어든다. 보고서 6.3의 "이직 공백 동안 실업급여 없음"과 다르다. 새 실직에서는 uiLeft를 다시 정하므로 이 경우에만 생긴다. 26세가 30개월째 이직(공백 2개월)하는 프로필로 재보니 2,000경로 중 24개(1.2%)에서 생겼다.

Java는 일단 기준 구현대로 만든다 (정답 파일과 맞추기 위해). 고칠지는 9.3에서 정한다. 고치면 kernel.py·engine.js·golden.json·API 예시 값을 함께 바꿔야 한다.

9.3 팀이 정해야 할 것

항목	이 문서의 처리	정해야 할 것
9.2의 실업급여 동작	기준 구현대로	이직 공백이 시작될 때 uiLeft = 0으로 고칠지
변동지출 감액 탐색 범위	데모대로 5~60만 원(변동지출 이하)	API 예시의 searched 문구("월 여유자금까지")를 이에 맞출지

항목	이 문서의 처리	정해야 할 것
개선 포인트의 비상금 type 이름	EMERGENCY_FIRST (제안)	API 예시에 없는 이름이라 확정 필요
지출 DETAIL 합계	items 합 우선 (제안)	total과 다를 때 오류로 돌려줄지
경고 두 개가 동시에 해당	WARN_EXPENSE_OVER_INCOME이면 WARN_MONTHLY_SHORTFALL는 빼는 것을 제안 (데모도 하나만 표시)	둘 다 보낼지
eventMarkers (/simulate)	정의 없음	이벤트의 달과 이름을 그대로 돌려주는 것을 제안
가능성 범위 문장에서 low = high	같은 문장 ("대부분 0%~0% 사이예요")	두 값이 같을 때 따로 문장을 둘지 (화면 문구라 계산에는 영향 없음)